National Economics University
College of Technology

Travel Insurance Management System

TripAssure Insurance Co.

Course: Introduction to Databases
Final Project Report

Student: Le Minh Thong
Student ID: 11247227
Class: DS66B

Instructor: Dr. Tran Hung

May 2, 2026

Abstract

This report presents the design and implementation of a travel insurance management system for TripAssure Insurance Co., a fictional domestic insurer. The system manages the complete operational lifecycle of travel insurance: customer enrollment, contract issuance, claim processing, assessment, and payout disbursement.

The database comprises nine third-normal-form tables built on MySQL 8.0, supported by five indexes, four views, two user-defined functions, three stored procedures, three triggers, and a role-based access control scheme. A Streamlit web dashboard provides the operational interface for agents, assessors, and administrators, delegating all business logic to the database layer.

The system is distinguished from generic insurance implementations by treating the *trip* as an independent insurable entity with its own lifecycle, introducing geographic risk multipliers for premium calculation, and enforcing claim-type-specific coverage rules at the database level.

Contents

List of Figures iii

List of Tables iv

1 System Analysis and Requirements 1
- 1.1 Business Context . . . 1
  - 1.1.1 How Insurance Works . . . 1
  - 1.1.2 Travel Insurance Specifically . . . 1
- 1.2 System Scope . . . 2
- 1.3 Functional Requirements . . . 2
- 1.4 Business Rules . . . 3

2 Database Design and Implementation 4
- 2.1 Entity Identification . . . 4
- 2.2 Entity-Relationship Diagram . . . 6
- 2.3 Normalization Analysis . . . 7
- 2.4 Schema Implementation . . . 7

3 Advanced Database Objects and Optimization 9
- 3.1 Indexes . . . 9
- 3.2 User-Defined Functions . . . 9
- 3.3 Views . . . 10
- 3.4 Stored Procedures . . . 10
- 3.5 Triggers . . . 10
- 3.6 Security and Access Control . . . 11

4 Python Application 13
- 4.1 Architecture . . . 13
- 4.2 Transaction Management . . . 13
- 4.3 Functional Modules . . . 14
- 4.4 Application Interface: Execution Evidence . . . 14

5 Conclusion and Recommendations 17
- 5.1 Summary . . . 17
- 5.2 Current Limitations . . . 17
- 5.3 Recommendations . . . 17

References 19

List of Figures

1.1 Core lifecycle of a travel insurance contract 1

2.1 Entity-relationship diagram, TripAssure Insurance System 6

3.1 Automated event chain across the three triggers 11

4.1 Application architecture — thick database, thin application 13
4.2 Contract initialization 15
4.3 Policy enforcement 15
4.4 Automated disbursement 16
4.5 Financial reporting 16

List of Tables

1.1 System actors and their responsibilities 2

2.1 Entity classification by dependency level 5
2.2 3NF verification summary 7

3.1 Index definitions and query patterns optimized 9
3.2 View definitions and intended consumers 10
3.3 Role-based access control summary 11

4.1 Python modules and database objects invoked 14

Chapter 1

System Analysis and Requirements

1.1 Business Context

1.1.1 How Insurance Works

Insurance is a financial mechanism that redistributes risk across a population. Each participant pays a small, predictable *premium* in exchange for protection against a larger, uncertain loss. The insurer pools premiums from many policyholders and compensates those who file valid claims. Commercial viability depends on total premiums collected exceeding total claims paid—maintained through actuarial pricing and fraud controls.

Every insurance product follows the same core lifecycle: a customer purchases coverage, an insurable event may or may not occur during the coverage period, and if it does, a claim is filed, assessed, and either paid out or rejected. Figure 1.1 illustrates this lifecycle.

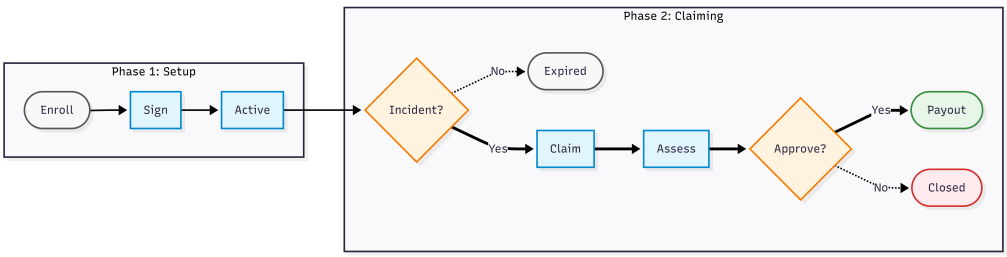

Figure 1.1: Core lifecycle of a travel insurance contract

1.1.2 Travel Insurance Specifically

Travel insurance applies this model to a single time-bound event: a trip. The insured object is not a durable asset—it is *an event with a defined start date, end date, and geographic scope*. When the trip ends, coverage terminates automatically.

Illustrative example. A customer books flights to Tokyo for two travelers (1–8 September) with total non-refundable costs of 12,000,000 VND. She purchases a travel insurance plan for 945,000 VND. Three weeks before departure, she is hospitalized. Without insurance, the booking cost is forfeited. With insurance, she files a *trip cancellation* claim and receives 80% compensation— approximately 9,600,000 VND.

Two structural properties of this domain directly drive the database design:

- **Premium is multivariable.** The cost of coverage depends on the destination's risk profile, the number of travelers, and trip duration. Medical costs in Tokyo differ significantly from those in Đà Lạt.
- **Claim types carry distinct compensation rules.** Trip cancellation, medical emergency, and baggage loss are governed by different reimbursement percentages and maximum payout ceilings. These rules must reside in the database, not be hard-coded in application logic.

1.2 System Scope

TripAssure is an *internal operations platform*, not a customer-facing portal. Insurance agents, claim assessors, and administrators interact with the system to manage contracts and claims on behalf of customers.

Table 1.1: System actors and their responsibilities

Actor	Responsibilities	Restrictions
Insurance Agent	Enroll customers; create trips and contracts; file claims on behalf of customers	Cannot approve or reject claims
Claim Assessor	Review pending claims; record assessment decisions and approved amounts	Cannot create contracts or modify customer data
Administrator	Full system access; financial reporting; user and catalog management	—

1.3 Functional Requirements

Contract lifecycle management

- Register customers with `NationalID` as unique identifier
- Record trip details: destination, travel dates, traveler count
- Calculate premium automatically from plan base rate, trip duration, and destination risk multiplier
- Issue contracts linking customer, trip, and plan; lock premium at signing
- Synchronize contract status with trip lifecycle automatically

Claims and payouts

- Accept claims only against active contracts
- Enforce annual claim limits per contract to prevent abuse
- Route claims to assessors with full contextual information
- Generate payout records automatically upon assessment approval—no manual entry
- Support reporting on total payouts, claim approval rates, and contract performance

1.4 Business Rules

The following rules are enforced at the database layer and cannot be bypassed regardless of access method.

1. **Claims require an active contract.** A claim may only be filed against a contract with status `active`.
2. **Annual claim limits apply.** The number of claims per contract per calendar year cannot exceed the plan's `MaxClaimsPerYear`.
3. **Payouts are trigger-generated only.** A payout record is created if and only if an assessment decision is `approved`. Direct insertion into `Payouts` is prohibited by role permissions.
4. **Contract status tracks trip lifecycle.** When a trip is marked `completed`, its contract transitions to `expired`; when cancelled, the contract becomes `cancelled`.
5. **Premium is frozen at signing.** `TotalPremium` is stored as a fixed value; changes to the underlying plan's base rate do not affect existing contracts.

Chapter 2

Database Design and Implementation

2.1 Entity Identification

Entity identification translates business workflows into a physical data model. Each real-world object the system tracks independently dictates a candidate table. Table 2.1 groups these entities by dependency level, establishing the explicit creation order required to satisfy foreign key constraints.
Five of these entities represent deliberate deviations from the project's suggested five-table schema, justified as follows:

DestinationRegions. Premium multipliers vary because emergency medical costs differ significantly across regions (e.g., East Asia vs. Europe). Extracting this multiplier data into a separate *region* table prevents transitive dependencies inside `Trips`, thereby preserving 3NF.

Trips. A trip has an independent lifecycle (`upcoming` → `ongoing` → `completed`) that must be tracked separately from its contract. This separation is required to enable trigger-based status synchronization and to enforce the one-trip-one-contract rule via `TripID UNIQUE`.

ClaimTypes. Compensation rules (`CoveragePercentage`, `MaxCoverageAmount`) vary by incident category and require periodic updates. Because an `ENUM` column stores only labels and cannot accommodate per-value rules, a separate table is necessary to allow standard policy updates without altering the schema.

Assessments linked to Claims. While a single contract may generate multiple claims, each claim requires exactly one assessment. Linking `Assessments` directly to `ClaimID` accurately reflects this 1:1 relationship, and the `UNIQUE(ClaimID)` constraint ensures a claim is never assessed twice.

TotalPremium as a frozen value. Although `TotalPremium` can be derived from plan and trip data (making it technically redundant), insurance contracts are legal documents. This deliberate denormalization ensures that the premium agreed upon at signing remains fixed, even if the plan's base rate changes later.

Table 2.1: Entity classification by dependency level

Entity	Role in the domain	Depends on
Independent entities — no foreign keys to other domain tables		
`Customers`	Policyholder identity and contact data	—
`DestinationRegions`	Geographic risk zones with premium multipliers	—
`InsurancePlans`	Product catalog with pricing and claim limits	—
`ClaimTypes`	Incident categories with coverage rules	—
Dependent entities — reference one or more tables above		
`Trips`	The insured event; linked to a risk region	DestinationRegions
`Contracts`	Binding agreement: customer × trip × plan	Customers, Trips, InsurancePlans
`Claims`	Compensation request against a contract	Contracts, ClaimTypes
`Assessments`	Assessor's formal decision on a claim	Claims
`Payouts`	Disbursement record; trigger-generated only	Claims

2.2 Entity-Relationship Diagram

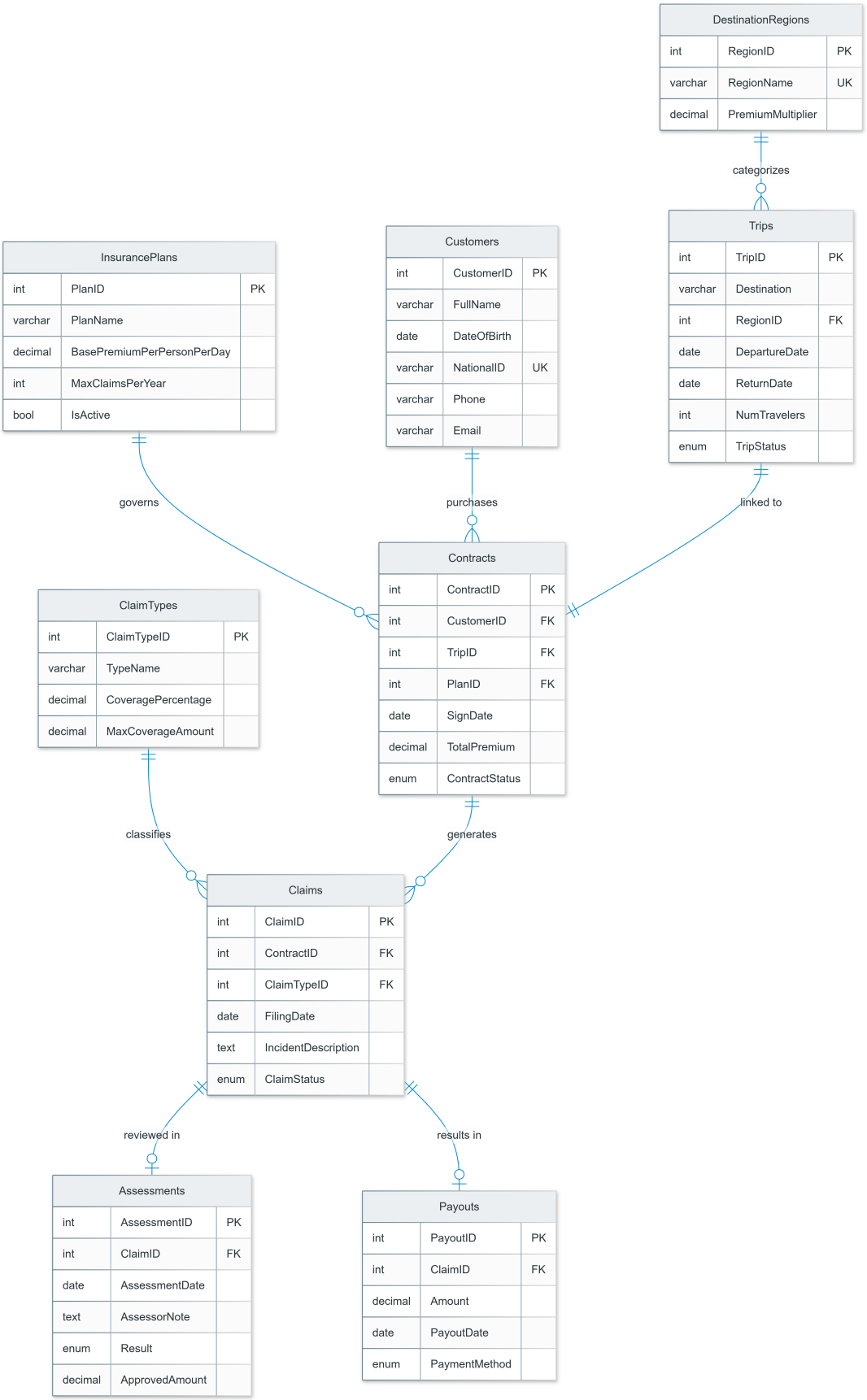

Figure 2.1: Entity-relationship diagram, TripAssure Insurance System

2.3 Normalization Analysis

All nine tables are verified against third normal form (3NF). The notation $A \to B$ denotes a functional dependency: knowing A uniquely determines B. Table 2.2 summarises the result; the one exception is annotated with a footnote.

Table 2.2: 3NF verification summary

Table	Key functional dependencies	3NF
`Customers`	`CustomerID` → all attributes `NationalID` → `CustomerID` (candidate key)	✓
`DestinationRegions`	`RegionID` → all attributes `RegionName` → `PremiumMultiplier` (both are candidate keys; not a transitive dep.)	✓
`InsurancePlans`	`PlanID` → all attributes	✓
`ClaimTypes`	`ClaimTypeID` → all attributes	✓
`Trips`	`TripID` → all attributes `RegionID` is a FK; does not determine other non-prime attributes	✓
`Contracts`	`ContractID` → all attributes `TotalPremium`: intentional denormalization*	✓*
`Claims`	`ClaimID` → all attributes	✓
`Assessments`	`AssessmentID` → all attributes `ClaimID` → `AssessmentID` (alternate key, 1:1)	✓
`Payouts`	`PayoutID` → all attributes `ClaimID` → `PayoutID` (alternate key, 1:1)	✓

* `TotalPremium` is a derived attribute stored deliberately: premium must remain immutable after signing even if the underlying plan rate changes. See Section 2.1 for full justification.

2.4 Schema Implementation

The DDL below highlights constraints that encode the design decisions established in Sections 2.1 and 2.3.
Full DDL is provided in `schema.sql`.

```sql
-- Trips: date integrity enforced at DB level
CREATE TABLE Trips (
    TripID          INT             AUTO_INCREMENT PRIMARY KEY,
    DepartureDate   DATE            NOT NULL,
    ReturnDate      DATE            NOT NULL,
    TripStatus      ENUM('upcoming','ongoing','completed','cancelled')
                                    NOT NULL DEFAULT 'upcoming',
    CONSTRAINT chk_trip_dates CHECK (ReturnDate > DepartureDate)
);

-- Contracts: 1:1 with Trips prevents double-insuring one trip
CREATE TABLE Contracts (
    ContractID      INT             AUTO_INCREMENT PRIMARY KEY,
    TripID          INT             NOT NULL UNIQUE,
    TotalPremium    DECIMAL(10,2)   NOT NULL    -- frozen at signing
);

-- ClaimTypes: coverage rules per type, not hard-coded in application
CREATE TABLE ClaimTypes (
    ClaimTypeID         INT             AUTO_INCREMENT PRIMARY KEY,
    CoveragePercentage  DECIMAL(5,2)    NOT NULL
        CHECK (CoveragePercentage BETWEEN 0 AND 100),
    MaxCoverageAmount   DECIMAL(10,2)   NOT NULL
);
```

Listing 2.1: Selected DDL constraints from schema.sql

Chapter 3

Advanced Database Objects and Optimization

3.1 Indexes

Indexes are created on columns that appear in frequent `WHERE` and `JOIN` patterns on tables expected to grow large over time. Foreign key columns in MySQL InnoDB are *not* automatically indexed; explicit creation is required. These five indexes also serve as the primary performance-tuning mechanism for large-scale contract and claim queries.

Table 3.1: Index definitions and query patterns optimized

Index	Column	Query pattern
`idx_contracts_customer`	`Contracts.CustomerID`	Agent: all contracts for customer X
`idx_contracts_status`	`Contracts.ContractStatus`	Dashboard: active contracts filter
`idx_claims_contract`	`Claims.ContractID`	Assessor: all claims under contract Y
`idx_claims_status`	`Claims.ClaimStatus`	Claim queue: pending / under_review
`idx_trips_departure`	`Trips.DepartureDate`	Monthly reports: date-range filter

3.2 User-Defined Functions

UDFs are defined before views because two views reference them as inline computed columns—a capability stored procedures cannot provide.

fn_CalculatePremium(`PlanID, TripID`). Computes the insurance premium:

$P = \texttt{BasePremiumPerPersonPerDay} \times \texttt{NumTravelers} \times \texttt{TripDuration} \times \texttt{RegionMultiplier}$

Called within `sp_CreateContract` to produce `TotalPremium`, and from Python to preview the premium before a customer confirms.

fn_GetClaimSuccessRate(`ContractID`). Returns the percentage of approved claims for a contract:

$$R = \frac{\text{approved claims}}{\text{total claims}} \times 100$$

Used as a computed column in `vw_ContractClaimSummary`. Returns `0.00` when no claims exist.

3.3 Views

Each view encapsulates a multi-table join for a specific actor and use case, and serves as an access control boundary—actors query views rather than base tables directly.

Table 3.2: View definitions and intended consumers

View	Actor	Purpose
`vw_ActiveContracts`	Agent	Full contract context (customer, trip, plan) for the agent dashboard
`vw_PendingClaims`	Assessor	Claim queue filtered to `pending` / `under_review`; includes claim-type coverage rules inline
`vw_MonthlyPayoutSummary`	Admin	Payout totals aggregated by year and month
`vw_ContractClaimSummary`	Admin / Agent	Per-contract performance with approval rate via `fn_GetClaimSuccessRate()`

3.4 Stored Procedures

All three procedures wrap their operations in transactions with an `EXIT HANDLER` that rolls back on any exception and re-signals the error to the calling application.

sp_CreateContract. Validates two pre-conditions—plan is active, trip status is `upcoming`—then calls `fn_CalculatePremium` and inserts the contract atomically.

sp_FileClaim. Wraps the claim insert in a transaction so that any `SIGNAL` raised by the pre-insert trigger propagates cleanly to the Python layer.

sp_ProcessAssessment. Uses `SELECT ... FOR UPDATE` to lock the claim row before inspecting its status, preventing two assessors from simultaneously assessing the same claim. On success, inserting into `Assessments` activates the post-insert trigger.

3.5 Triggers

The three triggers form an automated event chain shown in Figure 3.1.

trg_before_claim_insert *(BEFORE INSERT on `Claims`)*
Enforces two business rules: the linked contract must be `active`, and the annual claim count must not exceed `MaxClaimsPerYear`. A BEFORE trigger is required because `SIGNAL SQLSTATE` prevents the write entirely; an AFTER trigger cannot undo a row already committed.

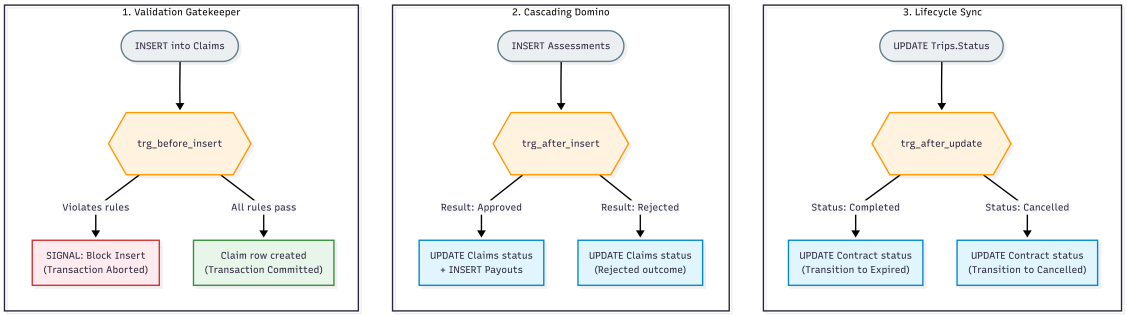

Figure 3.1: Automated event chain across the three triggers

trg_after_assessment_insert *(AFTER INSERT on `Assessments`)*
Reads `NEW.Result`: if `approved`, updates `ClaimStatus` and inserts a `Payouts` record automatically; if `rejected`, updates `ClaimStatus` only. This guarantees that a payout can exist only when backed by a formal approved assessment.

trg_after_trip_status_update *(AFTER UPDATE on `Trips`)*
Synchronizes `ContractStatus` when a trip reaches a terminal state. Guard conditions `NEW.TripStatus IN ('completed','cancelled')` and OLD.TripStatus NOT IN ('completed','cancelled') ensure the trigger fires exactly once per terminal transition.

3.6 Security and Access Control

Three database users map to the three operational roles, each granted the minimum permissions required (principle of least privilege). Database usernames use the `tripAssure_` prefix as an internal naming convention.

Table 3.3: Role-based access control summary

User	Objects accessible	Access
`tripAssure_agent`	`Customers`, `Trips`, `Contracts`, `Claims`, `InsurancePlans`, `ClaimTypes`, `DestinationRegions`, `vw_ActiveContracts`, `vw_ContractClaimSummary`	SELECT / INSERT
	`sp_CreateContract`, `sp_FileClaim`, `fn_CalculatePremium`	EXECUTE
`tripAssure_assessor`	`Claims`, `Contracts`, `Customers`, `ClaimTypes`, `vw_PendingClaims`	SELECT
	`Assessments`	INSERT
	`sp_ProcessAssessment`, `fn_GetClaimSuccessRate`	EXECUTE
`tripAssure_admin`	`TravelInsuranceDB.*`	ALL PRIVILEGES

No role may insert directly into `Payouts`; all payout records are created exclusively by

`trg_after_assessment_insert`.

Data protection. Sensitive customer fields (`NationalID`, `Phone`, `Email`) are readable only by `tripAssure_agent` and `tripAssure_admin`; the assessor role has no write access to customer data. For encryption at rest, MySQL's Transparent Data Encryption (TDE) can be enabled at the tablespace level without schema changes.

Backup and recovery. A daily `mysqldump` with `--single-transaction --routines --triggers` produces a consistent snapshot that includes stored procedures, functions, and triggers. Restoration is performed by piping the dump file back through the MySQL client.

Performance tuning. The five indexes in Table 3.1 eliminate full-table scans on the highest-traffic query patterns. `DECIMAL(10,2)` is used for all monetary columns in place of `FLOAT` to prevent binary rounding errors from accumulating across financial aggregations.

Chapter 4

Python Application

4.1 Architecture

The application follows a *thick-database, thin-application* pattern by design—not by omission. All business logic (validation, state transitions, calculations) is encapsulated in stored procedures, functions, and triggers. The web layer handles user interaction, calls procedures, and formats output. This ensures that business rules are enforced regardless of the access method, and that replacing the web interface with a REST API would require no changes to the database layer.

The system consists of two Python files: `app.py` provides the database connection layer and all query functions; `web.py` is the Streamlit web dashboard that imports from `app.py` and renders the role-based user interface.

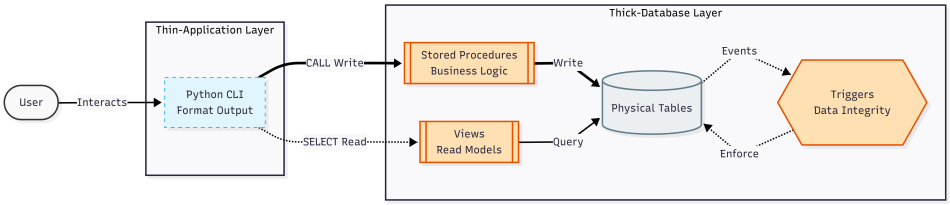

Figure 4.1: Application architecture — thick database, thin application

4.2 Transaction Management

All database calls in `app.py` use a `_cursor()` context manager that guarantees commit-on-success and rollback-on-exception without requiring explicit handling at each call site. Stored procedure calls use a dedicated `_sp_cursor()` variant that drains extra result sets with `nextset()` before reading `OUT` parameter variables.

```
@contextmanager
def _cursor():
    conn = get_connection()
    cur  = conn.cursor(dictionary=True)
    try:
        yield conn, cur
        conn.commit()
    except Exception:
        conn.rollback()
```

```
        raise
    finally:
        cur.close()
        conn.close()
```

Listing 4.1: _cursor() context manager in app.py

Any exception—including a `SIGNAL` raised by a trigger—causes automatic rollback and surfaces the error message to the web interface.

4.3 Functional Modules

Table 4.1: Python modules and database objects invoked

Module	Key functions	DB objects called
`enrollment`	`create_customer`, `find_customer`	Direct `INSERT` / `SELECT`
`contracts`	`create_trip`, `preview_premium`, `create_contract`	`sp_CreateContract`, `fn_CalculatePremium`, `vw_ActiveContracts`
`claims`	`file_claim`, `get_pending_claims`	`sp_FileClaim`, `vw_PendingClaims`
`assessment`	`process_assessment`, `get_claim_detail`	`sp_ProcessAssessment`
`reports`	Monthly payouts, success rates, expiry alerts	All four views

4.4 Application Interface: Execution Evidence

The web dashboard serves as a thin operational layer. The following captures demonstrate business rules and workflows being enforced entirely by the database engine based on user interactions through the Streamlit interface.

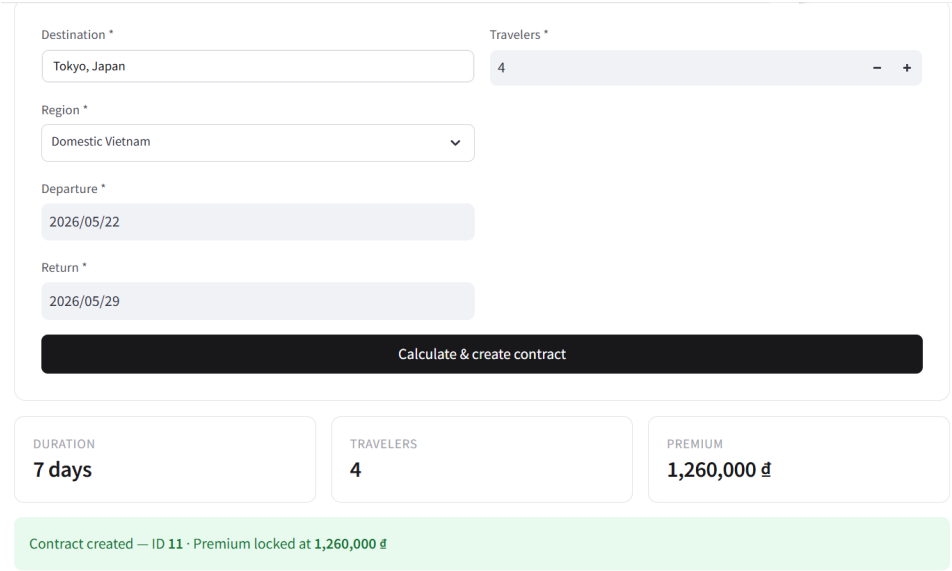

Figure 4.2: **Contract initialization.** An Agent inputs trip details to issue a new contract. The system queries `fn_CalculatePremium`, resulting in an instant, risk-adjusted premium calculation displayed to the user.

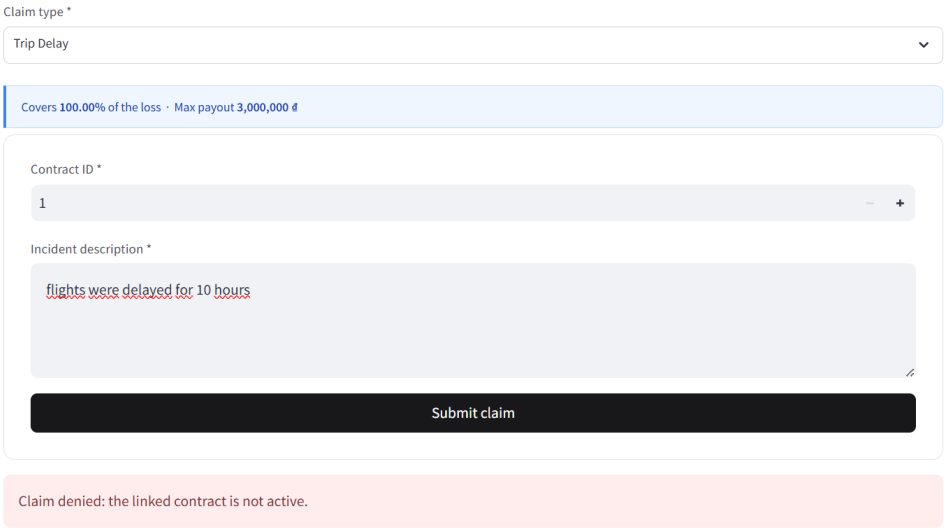

Figure 4.3: **Policy enforcement.** An Agent attempts to file a claim against an expired contract. The `trg_before_claim_insert` trigger intercepts the action, resulting in a hard database error that blocks the invalid submission.

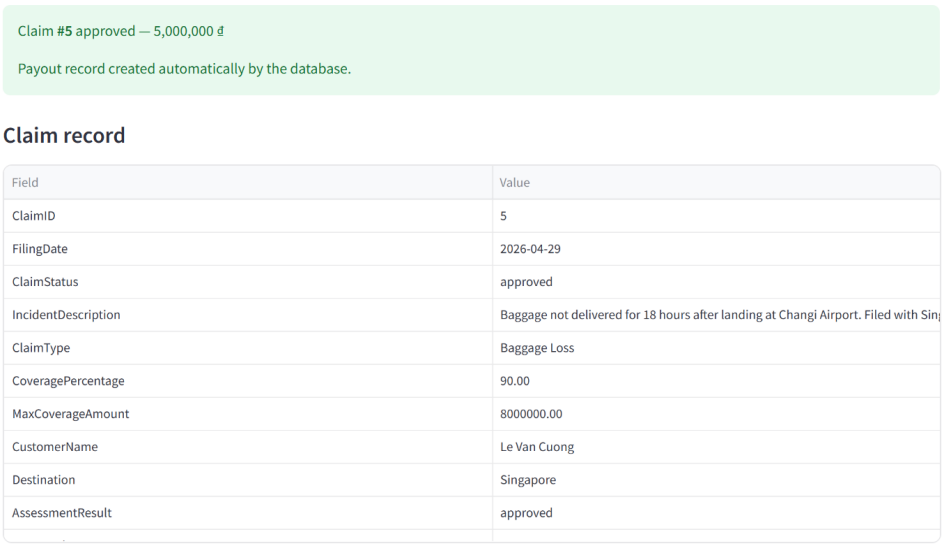

Figure 4.4: **Automated disbursement.** An Assessor formally approves a pending claim. This action activates the `trg_after_assessment_insert` trigger, resulting in the automatic generation of a financial payout record.

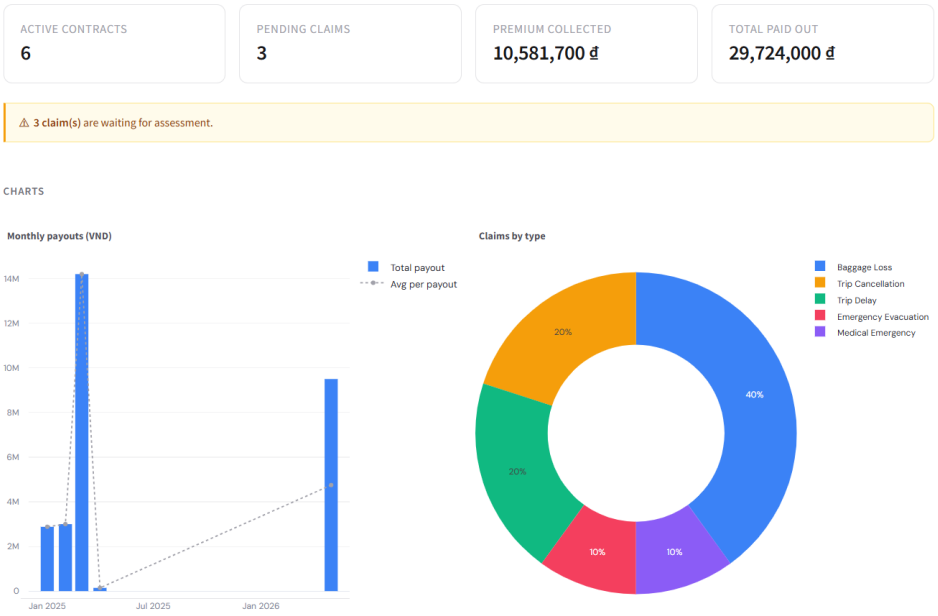

Figure 4.5: **Financial reporting.** An Administrator accesses the monthly dashboard. The system queries `vw_MonthlyPayoutSummary`, resulting in a real-time aggregated visualization of total claim payouts.

Chapter 5

Conclusion and Recommendations

5.1 Summary

The system covers the full operational lifecycle of travel insurance—from customer enrollment and contract issuance to claim assessment and automated payout generation. Nine 3NF tables, five indexes, four views, two UDFs, three stored procedures, three triggers, and three user roles were implemented in MySQL 8.0. Business rules are enforced entirely at the database layer, ensuring consistency regardless of the access method. The Streamlit web dashboard provides the operational interface using a thin-application pattern, delegating all logic to stored procedures.

5.2 Current Limitations

Interface.

- No customer-facing interface; agents file claims on behalf of customers.
- `TripStatus` is updated manually rather than driven by real-time date checks.

Financial.

- Payouts are recorded but not disbursed—no payment gateway integration.
- Single-currency (VND); no exchange-rate handling for international claims billed in foreign currency.

Data governance.

- No audit log for data modifications; historical changes are not tracked.
- Assessment decisions are immutable; reassessment requires a new claim record.

5.3 Recommendations

Customer self-service portal. A REST API (Flask or FastAPI) built on the existing stored procedures would allow customers to file claims directly. No schema changes are required.

Automated trip lifecycle management. A scheduled daily job querying `Trips` could update `TripStatus` based on the current date relative to `DepartureDate` and `ReturnDate`, eliminating manual updates.

Payment gateway integration. Connecting to VNPay or MoMo would initiate actual disbursements when a `Payouts` record is created—achievable via a webhook fired from `trg_after_assessment_insert`.

Multi-currency support. Adding an `OriginalCurrency` column and an `ExchangeRates` reference table would support accurate reimbursement for international medical claims billed in foreign currency.

Audit logging. History tables (`ContractsHistory`, `ClaimsHistory`) populated by AFTER UPDATE triggers would provide a full audit trail for regulatory compliance.

References

[1] Oracle Corporation, *MySQL 8.0 Reference Manual*, 2024. https://dev.mysql.com/doc/refman/8.0/en/

[2] A. Silberschatz, H. Korth, S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.

[3] Oracle Corporation, *MySQL Connector/Python Developer Guide*, 2024. https://dev.mysql.com/doc/connector-python/en/

[4] S. Astanin, *tabulate – Pretty-print tabular data in Python*, 2023. https://pypi.org/project/tabulate/